Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

## Operating Systems

# Laboratory Work No. 10

## Add Your Own System Call: trace

*Track B: Build Your Own Small OS (xv6)*

**Elaborated by:**

st. gr. FAF-241
Luchiciov Alexei

**Verified by:**

Reiţman Patricia

Chișinău – 2026

# Contents


1. Introduction ..... 3
2. User-Side Registration of trace ..... 3
   2.1. System Call Number and User Interface ..... 3
   2.2. First Check - Unknown System Call 23 ..... 3
3. Kernel-Side Implementation ..... 4
   3.1. Per-Process Mask and Handler ..... 4
   3.2. Syscall Table, Printing and Inheritance ..... 4
   3.3. Main Verification of trace ..... 5
4. Testing the Trace Masks ..... 5
   4.1. All System Calls ..... 5
   4.2. fork and Inherited Masks ..... 6
   4.3. write, open and close ..... 6
5. Observations ..... 7
6. Final Verification and Conclusion ..... 7

# 1. Introduction

The purpose of Laboratory Work No. 10 was to add a new **trace** system call to the **xv6** operating system. In the previous Linux laboratory, I observed system calls with **strace**; here I implemented a simplified tracing mechanism directly in the xv6 kernel. A process selects which calls to report using a bitmask, and the kernel prints the process ID, system call name, and return value.

The work was performed in the existing **~/xv6-riscv** tree inside an Ubuntu virtual machine and tested using **make qemu**. The recorded terminal outputs below come from my actual Lab 10 execution. This repository uses **SYS_trace = 23** (22 is already used by sync) and the kernel function **printk()**.

# 2. User-Side Registration of trace

## 2.1. System Call Number and User Interface

I first made a backup of the relevant files, then added the system call number, user-space declaration and assembly stub. I created **user/trace.c** to read the mask, call **trace()**, and replace its image with the requested command using **exec()**.

```
kernel/syscall.h:  #define SYS_trace 23
user/user.h:       int trace(int);
user/usys.pl:      entry("trace");
Makefile (UPROGS): $U/_trace\
```

```
int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

After updating **Makefile**, the terminal confirmed the entry at line **153**. The **trace** user program compiled successfully and was added to the xv6 disk image.

## 2.2. First Check - Unknown System Call 23

Before implementing the kernel handler, the expected output was an unknown-call error. The test produced this exact result twice, proving that the user-to-kernel system call path reached number 23:

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
$ trace 32 grep hello README
4 trace: unknown sys call 23
trace: failed
```

# 3. Kernel-Side Implementation

## 3.1. Per-Process Mask and Handler

In **kernel/proc.h**, I added **int trace_mask** to **struct proc**. This stores a bitmask for each process, so the tracing decision is process-specific. The new handler in **kernel/sysproc.c** reads the first argument using **argint(0, &mask)** and saves it in the current process.

```
// kernel/proc.h, inside struct proc
char name[16];
int trace_mask;

// kernel/sysproc.c
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

## 3.2. Syscall Table, Printing and Inheritance

In **kernel/syscall.c**, I declared **sys_trace()**, placed it in the dispatch table at **SYS_trace**, and added a name table for the system calls. Immediately after the kernel executes a call, the new condition examines its bit in the process mask and prints a trace line.

```
extern uint64 sys_trace(void);

// Part of the syscall dispatch table:
[SYS_trace] = sys_trace,

// After p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) & 1)
  printk("%d: syscall %s -> %ld\n",
         p->pid, syscallnames[num], p->trapframe->a0);
```

The **syscallnames[]** array includes all existing call names, such as **read**, **write**, **open** and **close**, together with the new **trace** entry. Because tracing happens *after* the call, the recorded value is the system call result stored in register **a0**.

In **kernel/proc.c**, I also copied the mask when creating a child process, so descendants retain the tracing configuration:

```
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
```

The terminal confirmed the modifications at **kernel/proc.h:104**, **kernel/proc.c:291**, **kernel/sysproc.c:119** and **kernel/syscall.c:160**. The rebuilt kernel compiled and booted successfully in QEMU.

### 3.3. Main Verification of trace

After completing the kernel changes, I ran the required command with mask **32**, which selects system call number 5, **read**. Unlike the earlier check, the new kernel printed the successful read operations instead of reporting unknown system call 23.

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

The three successful reads returned **1023 + 965 + 453 = 2441 bytes**, matching the recorded size of **README**. The final **read -> 0** shows end-of-file. The process PID is **3** in this run, although PIDs can vary between boots.

## 4. Testing the Trace Masks

### 4.1. All System Calls

I then enabled all bits from 0 through 30 with mask **2147483647**. The trace showed the setup call, execution of grep, opening README, its four reads and closing the descriptor.

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
```

This experiment confirmed that the names table and bit selection worked across different system calls. For this run, the process was PID **4**. The **trace -> 0** entry also confirmed that the newly added system call completed successfully.

### 4.2. fork and Inherited Masks

The command **trace 2 usertests forkforkfork** enables only bit 1, corresponding to **fork**. The test created many child processes, and tracing appeared under multiple process IDs, confirming that the **trace_mask** value was inherited. At the process-table limit, some fork calls correctly returned **-1**.

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 5: syscall fork -> 6
6: syscall fork -> 7
7: syscall fork -> 8
8: syscall fork -> 9
...
7: syscall fork -> 65
8: syscall fork -> 66
7: syscall fork -> -1
8: syscall fork -> -1
OK
ALL TESTS PASSED
```

The middle of the trace contained many additional successful forks, including lines from descendants with PIDs 7, 8, 10 and 56. The test ultimately reported **ALL TESTS PASSED**.

### 4.3. write, open and close

Mask **65536 = 2^16** selects only **write**. In the recorded run, **echo hi** printed the characters first and the newline separately; the kernel trace consequently appeared directly after **hi** and then after the newline.

```
$ trace 65536 echo hi
hi67: syscall write -> 2

67: syscall write -> 1
```

The final mask **2129920 = 2^15 + 2^21** enables **open** and **close** together, with no read lines. The observed output contained exactly those two call types:

```
$ trace 2129920 grep hello README
68: syscall open -> 3
68: syscall close -> 0
```

## 5. Observations

**1. Why does mask 32 report read, but not open or close?** Since **32 = 2^5** and system call number 5 is **SYS_read**, only the bit for read is set. The open (15) and close (21) bits remain clear.

**2. Why do several PIDs print fork traces?** The line **np->trace_mask = p->trace_mask;** inside **fork()** copies the mask from a parent process to each new child. Thus several descendants print their own fork calls.

**3. Why does the trace appear between "hi" and the newline?** The xv6 echo program made two **write()** calls: one for the two characters **hi** and another for the single newline. The kernel reports each return immediately after the corresponding write.

**4. Which mask selects only open and close?** With **SYS_open = 15** and **SYS_close = 21**, the correct value is **(1 << 15) | (1 << 21) = 32768 + 2097152 = 2129920**. The final observed test confirmed that choice.

## 6. Final Verification and Conclusion

The terminal confirmed that **SYS_trace** appeared in **kernel/syscall.h** and **kernel/syscall.c**, **sys_trace()** was present in **kernel/sysproc.c**, and the mask field, assignment and inheritance appeared in **proc.h**, **sysproc.c**, and **proc.c**. The executable source **user/trace.c** and **notes.txt** were present in the xv6 tree, and **Makefile** included the program.

```
$ grep -n 'SYS_trace' kernel/syscall.h kernel/syscall.c
kernel/syscall.h:24:#define SYS_trace 23
kernel/syscall.c:134:  [SYS_trace] = sys_trace,

$ grep -n 'trace_mask' kernel/proc.h kernel/proc.c \
    kernel/sysproc.c kernel/syscall.c
kernel/proc.h:104:  int trace_mask;
kernel/proc.c:291:  np->trace_mask = p->trace_mask;
kernel/sysproc.c:119:  myproc()->trace_mask = mask;
kernel/syscall.c:160:    if ((p->trace_mask >> num) & 1)
```

Some earlier commands were accidentally pasted as C statements directly into Bash, causing messages such as **"command not found"**. These did not modify the source: the subsequent Python edits succeeded, the kernel rebuilt, and all required runtime tests passed. The **cp -n** portability warning during backup was also non-fatal.

In conclusion, I registered system call number **23**, implemented a user-space wrapper and a kernel handler, added conditional **printk()** tracing and made tracing masks inheritable across **fork()**. The required README read sequence, all-call trace, fork stress test, write trace and combined open/close trace were successfully observed. This exercise clarified the complete path from a user-level system call to kernel dispatch and output.